C STORAGE CLASS

Programming API Perspective: C Storage Classes

- A Storage Class (SC) defines the scope, life-time, and physical location of a variable and precedes the variable name, e.g. *static int count*
  - *Auto*: the default storage class for all local variables
  - *Static*: used to keep a local variable in existence during the life-time of the program (instead of creating and destroying it each time the function is called and exits)
  - *Extern*: used to give a reference of a global variable that is visible to ALL the program files
  - *Register*: used to define local variables that **should** be stored in a register instead of RAM

Auto and Static Demonstration

Code

```
1  void example_func() {
2    static int static_v = 5;
3    int auto_v = 3;
4
5    static_v++;
6    auto_v++;
7    printf("static variable = %d, auto variable = %d\n\n",
         static_v, auto_v);
8  }
9
10 int main() {
11   example_func();
12   example_func();
13   return 0;
14 }
```

Output

```
static variable = 6, auto variable = 4

static variable = 7, auto variable = 4
```

Note:

- static variable start at 5
- auto variable start at 3

Auto and Extern Demonstration

Code

```
1  int extern_v = 10;
2  int v = 5;
3
4  void main() {
5    int v;
6    extern int extern_v;
7
8    printf("v = %d, extern_v = %d\n", v, extern_v);
9  }
```

Output

v = 25864, extern_v = 10

Note:

- V is random since we declare it without initial value.

- extern_v is become the same as the one initialized before.

Position Independent Executable (PIE)

- **PIE allows an executable to be loaded at a different virtual address each time it runs.**
- Instead of relying heavily on **fixed absolute addresses**, PIE uses **Position-independent / relative addressing** using offset from pointer.